



Clearing Hetero- geneous Compute

Zach Kelling

Hanzo AI

2026

Abstract

An exchange for compute is not an exchange for a commodity. The thing being traded is a bundle — device, memory, bandwidth, residency, locality, and the right to a deadline — and the bundles are not substitutes for each other except under constraints that vary per job. Maintaining a separate order book per device class throws away the structure that matters, and quoting a single price per accelerator-hour throws away all of it.

This paper is the mechanics. We define the job and bundle types, state the feasibility constraint, give the clearing objective including the residency terms, and show why the natural objective of minimising price is wrong in a way that is not marginal. We then define four order types — including a rent on state, which has no analogue in a stateless market — describe how the KV cache behaves as perishable inventory, and map each failure mode to a mechanism. We close by identifying which parts of this we already run in open source and can be lifted directly.

Contents

1	Jobs and bundles	3
2	The objective, and why cheapest is wrong	3
3	Order types	3
3.1	Intent	3
3.2	Reserve	4
3.3	Rent on state	4
3.4	Search	4
4	Cache as perishable inventory	4
5	Failure modes and mechanisms	4
6	What exists and can be lifted	5
7	Sequencing	6

1 Jobs and bundles

Definition 1 (Job). $J = (m, c, \Delta, \kappa, \tau, V)$ with m the model or computation required, c the context or input size, Δ the deadline, κ the determinism class, τ the required verification tier, and V the buyer's value at risk.

Definition 2 (Bundle). $R = (d, M, B, \beta, \alpha, \ell_0)$ with d the device, M available accelerator memory, B ingress bandwidth, $\beta \in \{0, 1\}$ whether the base model for m is resident, $\alpha \in \{0, 1\}$ whether the buyer's adapter is resident, and ℓ_0 the network round-trip to the buyer.

Feasibility is a hard constraint and must be evaluated before price:

$$R \models \text{cap}(J) \iff M \geq |w_m| + |\text{kv}(c)| \wedge c \leq c_{\max}(d) \wedge \kappa \in \mathcal{K}(d) \wedge \tau \in \mathcal{T}(R) \wedge \hat{t}(J, R) \leq \Delta. \quad (1)$$

The last conjunct is where residency enters feasibility rather than cost:

$$\hat{t}(J, R) = \ell_0 + (1 - \beta) \frac{|w_m|}{B} + (1 - \alpha) \frac{|a|}{B} + t_{\text{compute}}. \quad (2)$$

2 The objective, and why cheapest is wrong

$$R^* = \arg \min_{R \models \text{cap}(J)} \left[p(R) t_{\text{compute}} + \lambda_L \ell + \lambda_F \phi + \lambda_V v(\tau) + \lambda_D \delta + \mu_1(1 - \beta) + \mu_2(1 - \alpha) \right], \quad (3)$$

with $\mu_1/\mu_2 \approx 10^3$, the ratio of base-model size to adapter size.

Proposition 1. *Minimising $p(R)$ alone is not an approximation to (3); it can select an infeasible bundle.*

Take a job requiring a 140 GB base under a 3s deadline. A device at 0.50 h^{-1} without the model resident, on a 10 Gbit s^{-1} link, needs 112s to fetch the weights. It is not expensive relative to a device at 4 h^{-1} holding the model — it is outside the feasible set, by a factor of 37 on the binding constraint. Any allocator that ranks by price and then filters will spend most of its time rejecting its own top choices.

Two practical consequences. Feasibility must be evaluated first, and residency must be part of it. And μ_1 is not a smooth penalty — near an interactive deadline it behaves as a step, which is why a market for interactive inference looks less like a spot market and more like a market for scheduled capacity with warm state.

3 Order types

3.1 Intent

The buyer states what it wants, not what to buy:

```
intent:
  want: "best answer to <request>"
  deadline: 4s
  max_price: 0.08
  min_tier: T2
  determinism: D_inf
```

The exchange solves (3). This is the order type agents should use, and the reason is not convenience: an agent reasoning about device specifications is an agent whose behaviour breaks when the fleet changes.

3.2 Reserve

A commitment to capacity over a window, priced below spot in exchange for the provider’s ability to plan. Standard, and it works here for the usual reasons.

3.3 Rent on state

New, and specific to this market. The buyer pays a continuous fee to keep its adapter — or its model, or a conversation prefix — resident somewhere capable.

$$\text{rent} = \rho \cdot |a| \cdot dt \quad \text{buys} \quad \alpha = 1 \text{ with probability } \geq 1 - \epsilon \text{ over the term.} \quad (4)$$

The buyer converts a stochastic cold-start penalty into a predictable stream. The provider receives payment for memory it would otherwise hold speculatively or evict. This is an option on locality, it is continuously priceable, and it does not exist in a stateless compute market because there is no state to rent.

For the provider it also resolves a real scheduling problem. An accelerator holding k resident adapters faces a knapsack decision on every arrival, and the shadow price of a residency slot is the expected discounted value of its occupant. Rent makes that shadow price observable instead of guessed.

3.4 Search

For Las Vegas workloads: a domain, a predicate, a success probability, a deadline. Priced on expected work rather than measured time, settled on the first valid witness. Treated fully in the companion paper.

4 Cache as perishable inventory

Prefix caching means a conversation’s accumulated key-value state has value that decays — with time, as memory pressure evicts it, and with the conversation’s own probability of continuing. Writing γ for the per-turn continuation probability and s for the saving from a cache hit, the expected value of retaining a prefix for one more turn is γs , against a holding cost equal to the shadow price of the memory.

This is an inventory problem with a known solution shape, and it is the second place — after adapters — where a continuous formulation earns its keep over discrete order books. The quantity being priced varies smoothly, decays predictably, and has a clean opportunity cost.

5 Failure modes and mechanisms

Failure	Mechanism	Note
Provider defects mid-job	escrow, deadline refund	no adjudication needed
Deadline missed	automatic refund on timestamp	signing policy, not manual
Wrong output, class D	byte equality	proof of misbehaviour
Wrong output, class S	tier per job	buyer chooses
Residency misreported	self-revealing	the deadline is missed
Correlated committee	diversity as a constraint	otherwise m collapses to 1
Price manipulation	many providers, open quotes	thin markets stay thin

Table 1: Failure modes and the mechanism that addresses each.

Residency deserves emphasis because it is unusually clean. A provider cannot profitably claim to hold weights it does not hold, since the claim is falsified by the deadline it then misses. The market needs no attestation for this, which is rare and should be exploited rather than replaced with a certificate.

6 What exists and can be lifted

Two components are in production, open source, and directly reusable.

A Hamiltonian market maker. The pricing layer of equation (3) is implemented: roughly forty-seven hundred lines of Rust carrying a hidden Markov core for regime detection, symplectic price dynamics under a regime-dependent anharmonic potential, a compute-pricing adapter that modulates by service-objective tightness and quality tier, per-resource market states whose prices never cross, one-bit per-tenant preference adapters, and expected-free-energy routing. Quotes are deterministic given a job and market state. The companion paper on the market mechanism treats it in full; the point here is that the objective above is not a proposal.

A learned per-request router over frontier arms. The upper allocation problem — which intelligence, not which resources — is solved by a router that featurises a request in sub-microsecond time, scores every candidate arm by a bilinear utility $u(x, p) = x^\top W p$, and returns the argmax subject to hard ceilings on latency, cost and quality, with the weight matrix fit offline by ridge regression and adapted online per tenant by a contextual bandit. Its fan-out variant selects the top- N arms by the same utility, dispatches concurrently, and fuses candidates with a synthesizer that has no weights of its own — so a new arm is onboarded by adding a profile row rather than retraining a policy over the pool. Every request is metered on one plane, so cost is attributed per request to the arm that served it. This runs in production behind a single endpoint.

The structural point for a market: this is the same shape as equation (3) — hard feasibility gating a soft-weighted objective — at a different layer and over a different feature space. Intelligence allocation and capacity allocation are the same mechanism twice, and they compose because their interfaces are narrow: the upper layer asks for capacity meeting a specification without caring how price formed, the lower supplies a quote without caring which model runs.

Open weights are what a provider network can actually serve. A closed frontier arm reachable only through a vendor API cannot be scheduled onto a third-party accelerator; routing across such arms is a real product but the provider network is not doing the work. Open-weight models are the substrate a decentralized network can genuinely allocate — serve anywhere with the memory, fine-tune per tenant, distil, quantize. A market carrying both routes intelligence at the top and allocates real compute underneath.

A model catalog with tiered routing. Each model carries an ordered list of route rungs. Each rung names an upstream, a provider, a context limit, and two price pairs — retail and cost — so margin is explicit per rung rather than implied. Alternate arms and a synthesis route are first-class. The catalog validates its own invariants at load: a rung priced free may only route to a zero-priced upstream, a rung may not be configured to serve below cost, and every arm must name a known provider. Resolution maps a requested name to a concrete model and identifier.

This is a working price-and-route layer over heterogeneous model supply. What it does not yet price is state locality — the μ terms of (3) — which is the extension this series argues for.

A request router. Classification maps a request to a task from extracted features — text, approximate token count, media presence, explicit task and modality hints — kept pure so that a learned classifier can replace the heuristic without touching policy. Downstream are policy, route, replica and ring, giving pool selection, replica choice and consistent hashing.

The classifier boundary matters for an exchange: allocation quality depends on correctly typing incoming demand, and a market that cannot improve its classifier without redeploying its policy will not improve it.

An inference engine with adapters and a training API. Low-rank and quantized low-rank adapters, multi-adapter co-residency, mixture-of-adapters, and a training loop exposed over the serving API. The gap — per-sequence adapter selection within a batch — is stated precisely in the companion paper on residency.

Settlement. Threshold signing across twenty or more chains with per-job escrow and no key reconstruction, behind a signer-agnostic assembler boundary. Treated in the companion paper on custody.

7 Sequencing

The order that gets a market to real verified volume soonest, rather than the order that looks most complete on a diagram: carry search volume first, because verification is free and settlement is unambiguous. Add deterministic-class inference next, because byte equality gives it verification at no marginal cost. Add algebraic spot-checking for the general inference case. Add residency pricing and state rent once adapters are per-request selectable, because until then the μ terms are unobservable. Add succinct proofs last, for the statements that settle on chain.

Every step in that order produces a market that works at the end of it, which is the property worth optimising for.